Basketball-Instants Dataset – Quick Reference Guide

1. Directory Tree

```
basketball-instants-dataset/
├── basketball-instants-dataset.json      # master list of "instants" (global ann
otation)
├── <ARENA_LABEL>/                  # 1 folder per arena (e.g. KS-FR-BLOIS)
│   └── <GAME_ID>/                 # 1 folder per televised game
│       ├── camcourt1_<ts>.json          # camera-1 calibration + 2-D detection
s
│       ├── camcourt2_<ts>.json          # camera-2 calibration + 2-D detection
s
│       ├── camcourt1_<ts>_0.png         # RGB image @ offset 0   (view-1)
│       ├── camcourt2_<ts>_0.png         # RGB image @ offset 0   (view-2)
│       ├── camcourt1_<ts>_40.png        # RGB image @ offset 40 ms
│       ├── camcourt2_<ts>_40.png        # ...
│       ├── camcourt1_<ts>_1000.png      # optional long-offset frame
│       ├── camcourt1_<ts>_humans.png    # optional human mask (fg segme
ntation)
│       └── ...
└── ...
```

`<ts>` is the UNIX time-stamp in milliseconds (matches the *timestamp* field in the JSONs).

---

2. Global File – `basketball-instants-dataset.json`

An **array** where each element describes *one multi-view instant* (≈1/25 s).

Key fields (abridged):

Field	Type	Description

`arena_label`	str	Arena folder name.
`game_id`	int	Game sub-folder.
`timestamp`	int	Base time-stamp used in file names.
`num_cameras`	int (=2)	Always two fixed cameras.
`offsets`	list[int]	Frame offsets saved for this instant, usually `[0, 40, 1000]` ms.
`annotation_game_state`	str	*standard_game*, *free_throw*, *throw_in*, ...
`annotations`	list[obj]	3-D ground-truth objects (see below).
`fgmask`	bool	Foreground mask (binary) exists.
`annotated_human_masks`	bool	If per-pixel human masks were drawn.
`annotator_id`, `annotation_ts`, `annotation_duration`	provenance info.	

`annotations` elements:

- **ball** – `{type:"ball", center:[x,y,z], image:(0/1), visible:bool}`
- **player** – `{type:"player", team:(0/1 opponent, 2 = ref/bench), head/hips/foot1/foot2:[x,y,z], headOrientation:rad, image:(camera idx)}`

All coordinates are expressed in the **common 3-D court frame** (z < 0 means above the floor).

---

3. Per-Camera File – `camcourt<view>_<ts>.json`

```
{
  "view": 1,
  "calibration": {
    "KK":  [fx, 0, cx, 0, fy, cy, 0, 0, 1],
    "R":   [9 elements],          # rotation (world→cam)
    "T":   [Tx, Ty, Tz],          # translation  (m)
    "kc":  [k1, k2, p1, p2, k3],    # distorsion
    "alpha": 0.0,
```

```
    "img_width": 1624,
    "img_height": 1234
  },
  "timestamp_detections": 1513710529019,
  "timestamp_0": 1513710529019,
  "timestamp_1": 1513710529059,
  "timestamp_2": 1513710530019,
  "players": [
    {"status":1, "level":0.46, "pos_feet":[x_px, y_px]}, ...
  ]
}
```

`players` *list contains* ***2-D feet detections*** *ranked by confidence* `level`. `status` *1 ⇒ kept by detector, 0 ⇒ filtered out.*

---

4. File-Name Convention

```
camcourt<cam>_<timestamp>_<offset>.png
        ↑1 or 2              ↑ms after base ts
```

- `_humans.png` suffix ⇒ binary mask (white = player pixels).
- Same stem without suffix (`.json`) ⇒ per-camera calibration.

---

5. Quick Dataset Stats

- Arenas 14
- Games ≈ 70
- Instants ≈ 240 000
- Cameras 2 fixed, 1624 × 1234 px
- Ball annotated ~ 70 % instants
- Human masks ~ 50 % instants (check `annotated_human_masks`)

---

6. Typical Research Tasks

Task	What to use
3-D ball trajectory	`annotations` (ball) + camera calibrations
Multi-view 3-D pose	3-D joint positions in `player` + images
2-D segmentation	RGB + `_humans.png` masks
Camera calibration sanity	Project 3-D feet onto images and compare with `players.pos_feet` detections
Game-state recognition	`annotation_game_state` labels

7. Minimal Loading Snippets (Python)

```
import json, pathlib, numpy as np, cv2

ds_root = pathlib.Path('basketball-instants-dataset')

# ---------- load global list ----------
instants = json.load(open(ds_root/'basketball-instants-dataset.json'))

# ---------- resolve files ----------
def files_for(inst, cam=1, offset=0):
    folder = ds_root/inst['arena_label']/str(inst['game_id'])
    stem   = f'camcourt{cam}_{inst["timestamp"]}'
    return {
        'rgb'   : folder/f'{stem}_{offset}.png',
        'mask'  : folder/f'{stem}_{offset}_humans.png',
        'meta'  : folder/f'{stem}.json'
    }

# ---------- projection utils ----------
def camera_matrix(meta):
    K = np.array(meta['calibration']['KK']).reshape(3,3)
    R = np.array(meta['calibration']['R']).reshape(3,3)
    T = np.array(meta['calibration']['T']).reshape(3,1)
```

```
    return K @ np.hstack([R, T])      # 3×4

def project(P, xyz):
    xyz1 = np.c_[xyz, np.ones(len(xyz))]
    xy   = (P @ xyz1.T).T
    return xy[:, :2] / xy[:, 2:3]
```

8. Caveats & Tips

Issue	Advice
Not every offset exists for every instant	Check file existence before loading.
Some masks/calibration files missing	Guard with `path.exists()`.
Distortion (`kc`) ≠ 0	Undistort points/images if high accuracy needed.
Team codes	0 & 1 = teams on court, 2 = refs/bench/crowd.
Z axis sign	Negative Z ⇒ above ground (use absolute value for height).

Cheat-Sheet

You want …	Look at …
Game-state label	`annotation_game_state`
3-D ball (m)	`annotations` list where `type=="ball"`
3-D players	entries where `type=="player"`
2-D feet detection	`camcourt*.json → players[i].pos_feet`
Camera intrinsics	`KK`, `kc`, `alpha`
Camera extrinsics	`R`, `T`, `poscam`